本体如何帮助 Agent 理解与处理业务



贯穿问题

这笔订单能退吗？
为什么还不能马上退款？

让 Agent 读懂业务含义，找到判断依据

OntoWeave（织义）

目录

四章正文，最后留时间回答建设与维护中的常见问题

01 **Agent 的业务理解难题** 工具调用成功以后，为什么还会答错

02 **本体的概念、技术与价值** 业务含义怎样表达、查询与复用

03 **OntoWeave 与建模方法** 平台怎么分工，模型怎样建立

04 **电商退货与退款模拟案例** 用三笔订单核对完整判断过程

FAQ **常见问题** 选型、建设、质量与维护

看完后，能判断何时值得用本体、Agent 怎样用、由谁建设和验证

第 1 章

Agent 的业务理解难题

01

这笔订单能退吗?
为什么还不能马上退款?

任务机制 ／ 业务歧义 ／ 方法与局限

Agent 怎样完成一个业务任务

用户：这笔订单能退吗？为什么还不能马上退款？（当前订单 A101）

01

理解请求

语言模型拆分目标

当前订单：A101

先判断能否申请退货

再解释为何不能退款

02

查业务事实

查询工具与客户输入

A101 属于 A 店

普通商品、声明未使用

签收：10-01 10:00

验收未开始、审批未申请

03

检索候选政策

相似内容同时出现

A 店：签收后七天内

B 店：付款后七天内

两份都写普通商品可退

需要辨清各自适用范围

04

理解政策并判断

关联含义与本次事实

A101 应用 A 店政策

10-06 拟申请，距签收 120h

按声明可申请退货

退款仍需验收及批准

05

解释与继续

语言模型组织回答

按声明可申请退货

退款前需验收与批准

本次未提交或退款

时间均为 2026 年、UTC+08:00，拟申请为 10-06 10:00。业务判断还要识别申请与退款的不同前提。

检索到相似政策后，还要辨清适用对象、时间含义和处理阶段

两份相似政策，两个判断

客服知识库同时找到了两份“七天内可退”政策，Agent 应该用哪一份？

本次订单：A 店 · A101 · 普通商品 · 声明未使用

付款 09-28 10:00　签收 10-01 10:00　拟申请 10-06 10:00（2026 年，UTC+08:00）

A 店 · RET-DEMO-v1

签收后七天内，可申请退货

普通商品、声明未使用，
从客户签收起算 0—168 小时。

距签收 **120 小时**，处于时间窗内。

本次适用：按声明可申请

B 店 · RET-B-DEMO-v1

付款后七天内，可申请退货

普通商品、声明未使用，
从付款起算 0—168 小时。

距付款 **192 小时**，已超过时间窗。

若错套此政策：会判为超时

Agent 需要理解：政策归哪家店、约束哪类申请、“七天”从哪里起算。相似文字不能替代适用性判断。

A101 属于 A 店；先确定适用政策，再按对应业务含义判断

怎样让 Agent 分清相似政策

围绕上一页的难点：商家是否匹配、时间含义是什么、判断的是哪个阶段

01

提示词与示例

明确比较商家、适用范围和起算点；用 A／B 政策对照示例，要求给出选择依据。

02

文档检索 RAG

连同正文取得商家、版本与出处；让 Agent 看到两份完整政策，而非孤立的“七天”。

03

微调

用相似政策的辨别样例学习比较与补问；当前具体政策仍从资料中取得。

04

业务工具

提供订单归属、付款与签收字段及说明；计算 192／120 小时，供已选政策使用。

05

专家澄清

适用范围或字段含义缺失、冲突时，请业务负责人确认；保留确认的依据。

06

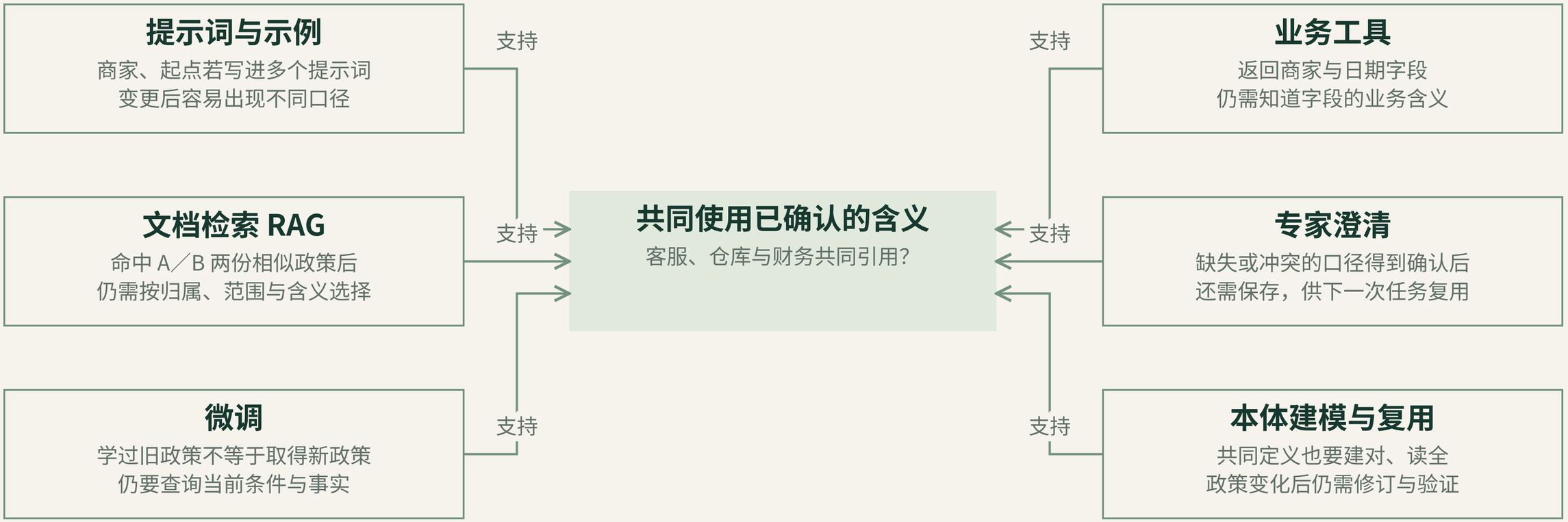
本体建模与复用

显式关联政策、商家、时间起点与申请条件，让不同任务引用相同定义。

把相似文字展开为可比较的业务含义，再选择适用政策；这些方法可以组合

不同方法的局限性

客服、仓库和财务，能否共同引用“申请、验收、退款”的含义?



共享文档和词典也能统一口径；是否需要本体，还要看关联与复用的价值

第 2 章

本体的概念、技术与价值

02

先说清业务里有什么、各自是什么意思、怎样关联，再讨论 Agent 如何使用。

定义与表达 ／ 使用机制 ／ 标准与适用性

什么是本体（Ontology）

为一块业务建立共同的概念、关系与约束，供人和程序使用

本体是一套可共享的领域定义：
明确有哪些概念、它们是什么意思、怎样关联，以及有哪些约束。

概念及含义

退货申请：客户请求退回
某个订单项。

关系

退货申请针对订单项。
政策适用于特定商家及商品范围。

条件与约束

申请资格与退款前提分开。
时间条件明确起点、单位和边界。

本体论讨论存在及其类别等哲学问题。
本演示的本体指知识工程中的 Ontology 模型，用来明确表达业务含义。

本体是知识模型；建模方法论指导怎样建立、验证与维护它

Palantir 如何将 Ontology 用于业务

一个产品化例子：让应用和 Agent 围绕共同的业务对象工作

Foundry 将分散的数据
连接到业务对象、关系与操作。

业务数据	订单表、商品数据、售后状态， 来自组织的不同系统。
业务含义	用订单、商品、退货申请等对象， 以及明确的属性与关系表达业务。
应用与操作	人员、应用和 Agent 通过获准能力， 查询对象并执行定义好的操作。

Palantir 是谁

2003 年成立的美国软件公司，
面向政府与企业提供数据分析及 AI 软件。

2017 · 项目应用

与空客合作推出
Skywise 航空数据平台。

2025 · 公司规模

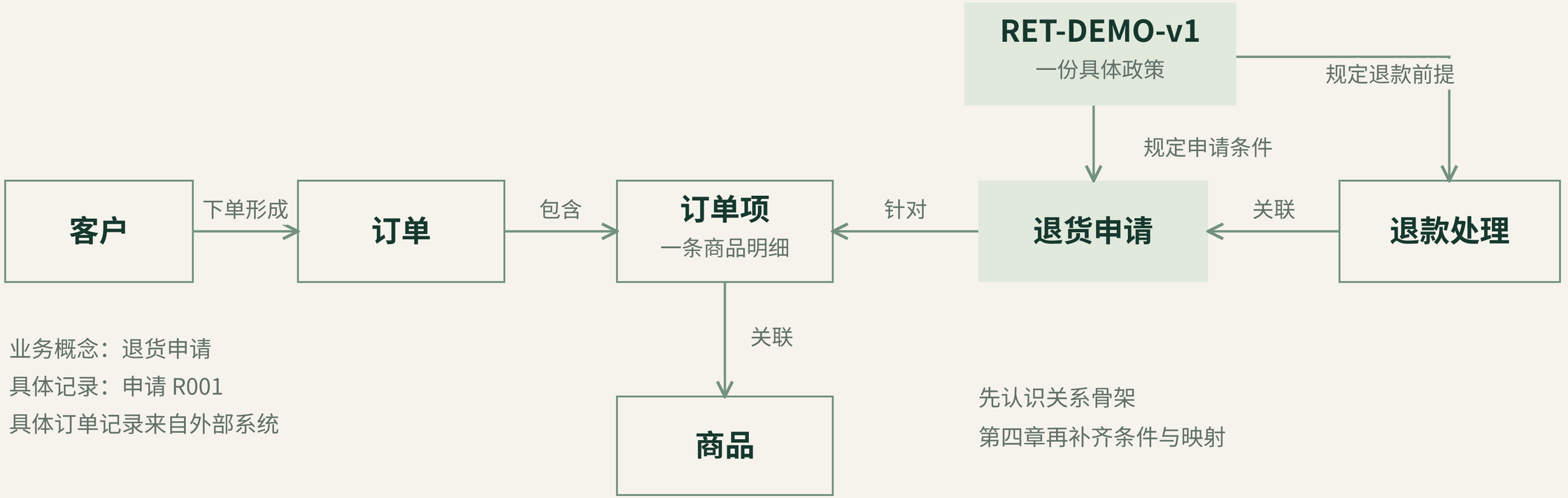
全年营收约 44.75 亿美元，
同比增长 56%。

来源：Airbus（2017）及 Palantir（2026-02-02）。
公司营收不代表 Ontology 单项技术的贡献。

Palantir 展示了一种产品实现；OntoWeave 的能力与业务执行边界在第三章说明

用本体描述退货业务

先认识对象和关系骨架，稍后补齐条件、事实来源与动作前提

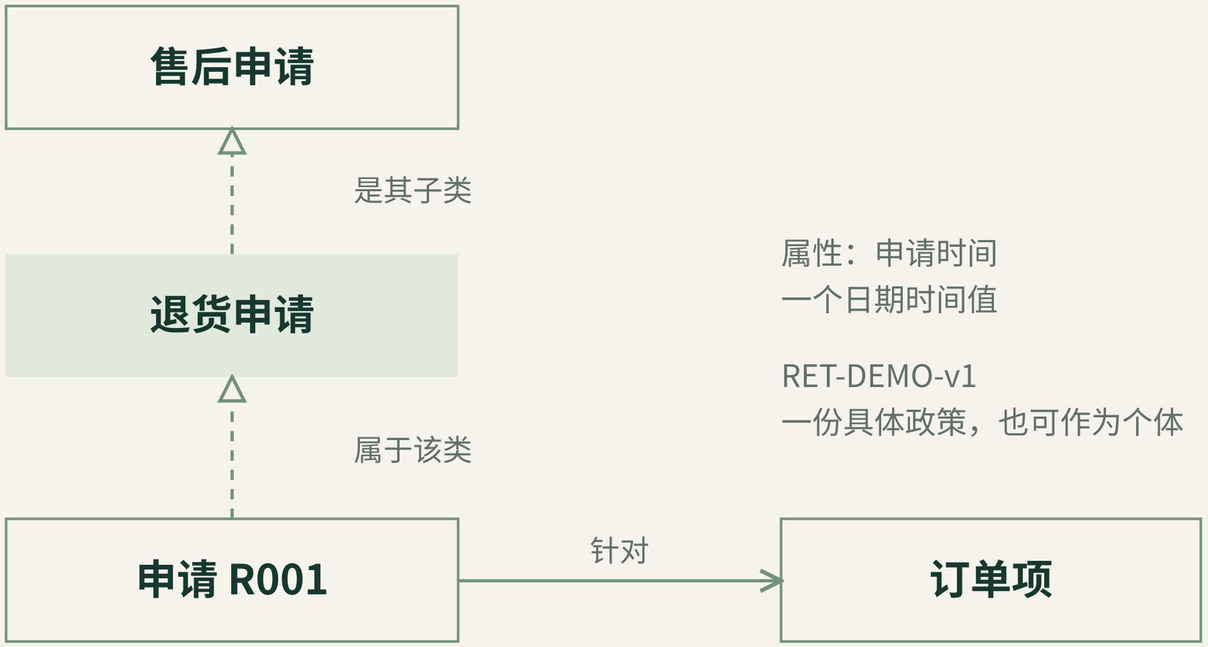


订单项是一条商品明细；“退货申请”这个概念与“申请 R001”这条记录分别表达

业务模型示意

本体中的基本表达

在同一小段业务中，读懂类别、个体、属性、关系和分类



类描述一类对象

个体指一个具体资源

属性连接值

关系连接资源

虚线表示分类／类型关系
实线表示业务关系
业务说明同时保留范围、例外与未知

名字相似不等于含义相同；同一资源需要稳定身份

本体、知识图谱与数据库的分工

同一业务，在定义、知识连接和记录存储三个层面表达

本体：约定含义与关系

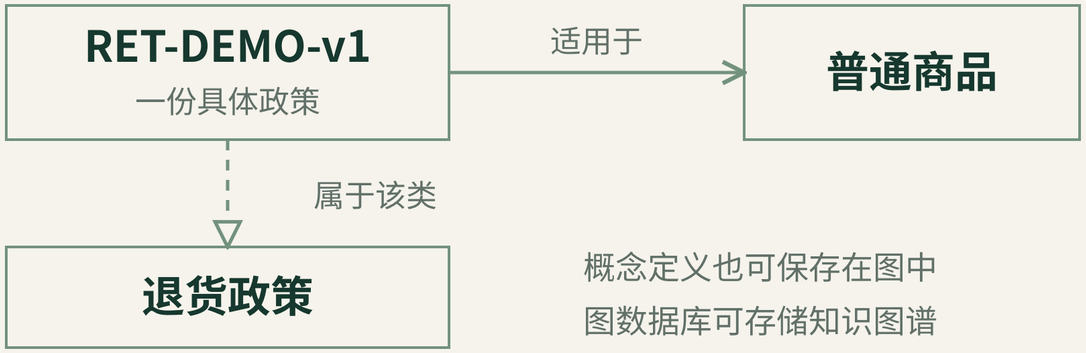
定义对象与可用关系



使用共同定义

映射到字段与连接

知识图谱：连接具体知识



概念定义也可保存在图中

图数据库可存储知识图谱

数据库：存储与查询记录

orders（虚构数据片段）

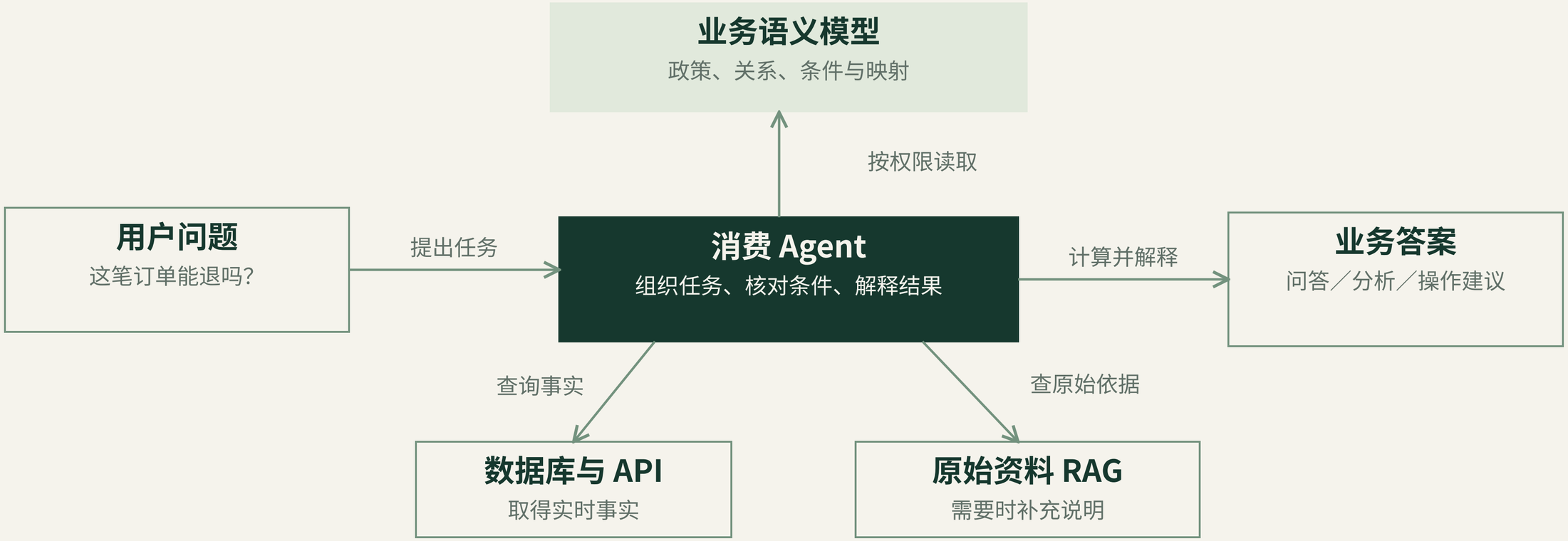
id	signed_at
A101	2026-10-01 10:00+08:00
A102	2026-09-27 10:00+08:00

业务行留在外部，schema 也可表达约束

本项目将订单等业务行保留在外部数据库，平台保存业务语义及映射

本体怎样帮助 Agent 工作

相关定义、关系和必要说明，合起来形成“语义上下文”



本体提供依据，Agent 组织任务与解释，外部系统提供事实并执行操作

业务含义怎样表达与查询

RDF、RDFS、OWL 与 SPARQL 各有职责

RDF 以“主体、谓词、客体”表达陈述



IRI：为资源提供可稳定引用的身份

Turtle：RDF 的一种书写格式

教学词汇示意，不代表专用业务规则执行合同

RDFS　基础类型与层级

`rdf:type`　属于某类

`rdfs:subClassOf`　子类

OWL　更丰富的形式语义

表达形式化公理及其含义

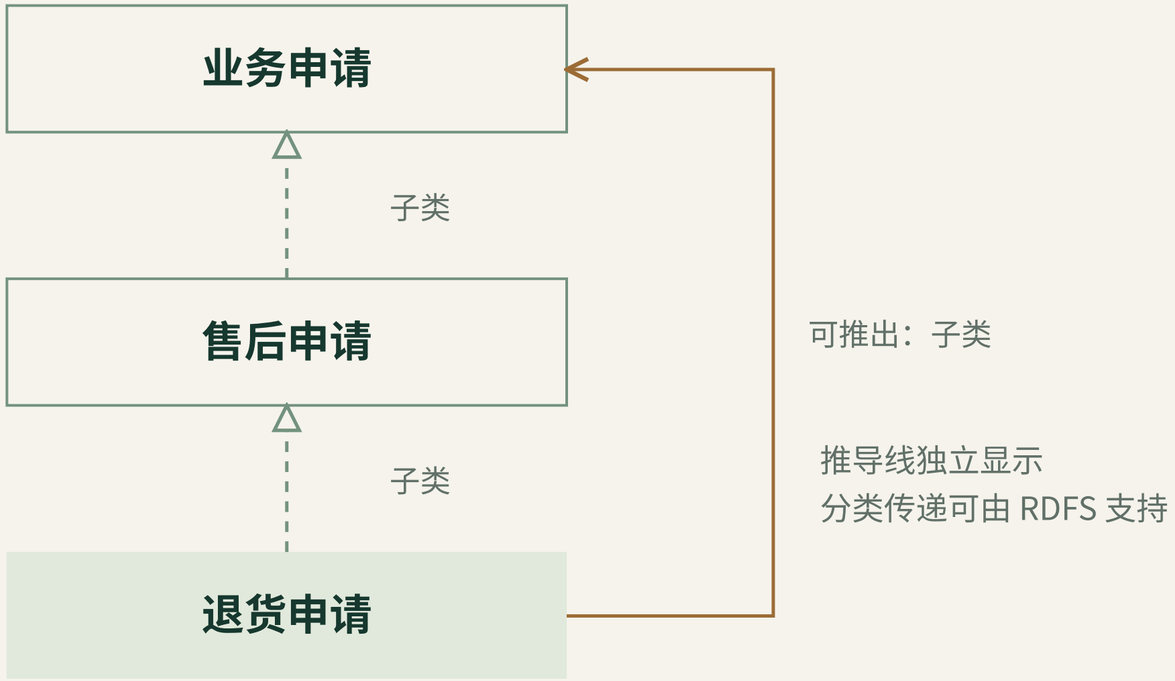
SPARQL　查询 RDF 图

找到符合图关系的资源与值

能机器读取业务说明，不等于其中所有条件已经形式化或能够自动执行

形式推理的作用与边界

由明确的形式前提推出隐含结论，与退款资格计算分别理解



原理示意

显式前提

形式规则

隐含结论

时间比较与金额计算需计算能力。
业务流程由对应系统执行。
缺少事实，不自动意味着事实为假。

OntoWeave 当前不提供自动推理运行

分类推导不能单独证明某张订单可以退款

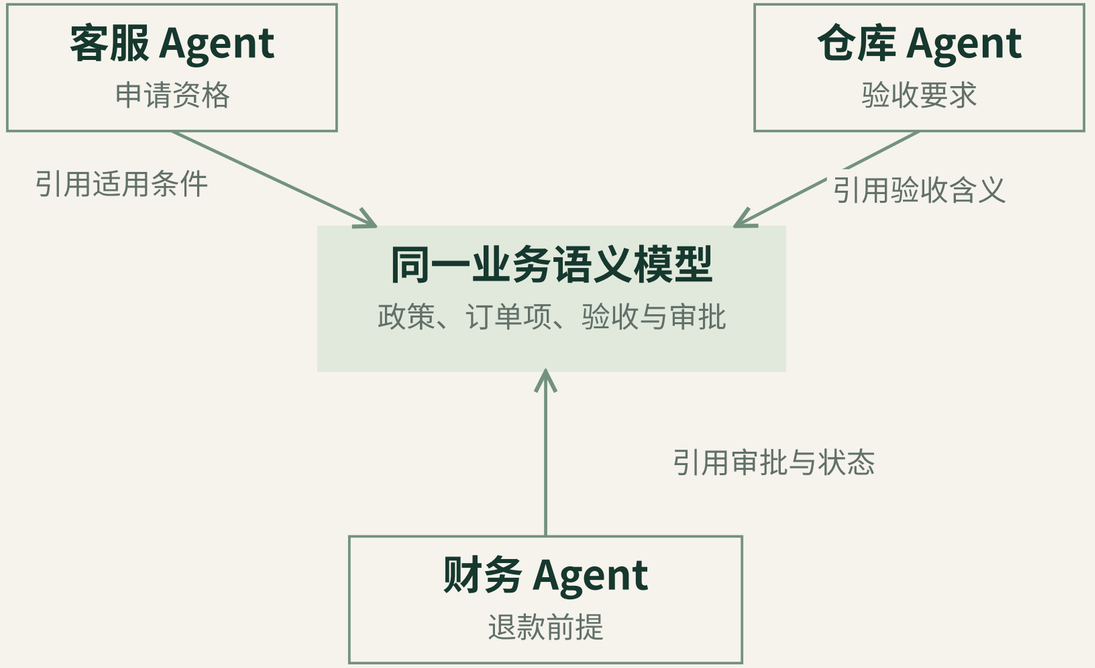
本体的优势与适用条件

用“为什么还不能退款”连接客服、仓库与财务的任务

共享口径已有轻量方案

统一文档、业务词典、
共享配置与规则服务，
都能帮助多个应用复用定义。

固定、简单的任务，可以继续使用更轻的方案。



值得建模的地方，在于跨对象、跨任务、跨系统的显式关联能否改善实际复用

第 3 章

OntoWeave 与建模方法

03

如何把业务材料变成可用的模型，
并让 Agent 可靠地取得它。

系统分工 ／ 建模方法 ／ 自动建模 ／ 读取验证

OntoWeave 的定位与责任边界

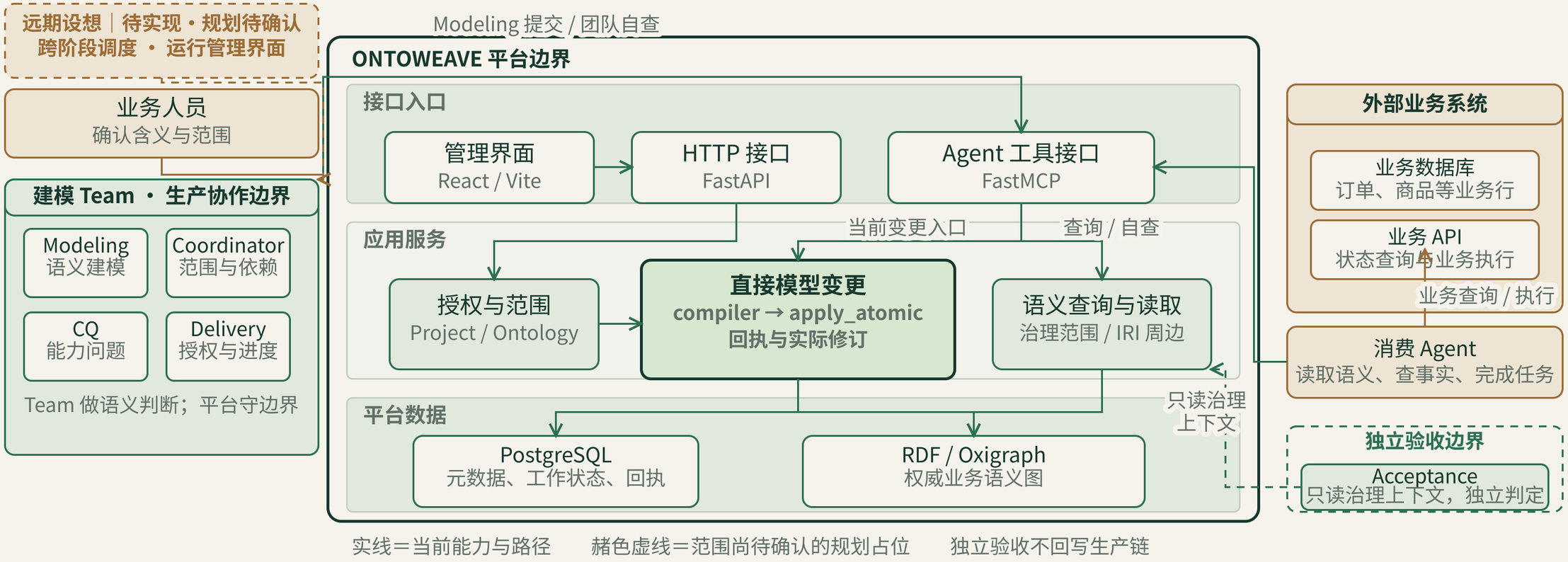
平台保存与提供业务语义，业务判断和业务执行各有负责人



平台检查可执行性与资源完整性，业务含义仍由建模与独立验证确认

系统架构

建模协作、平台服务、外部业务系统与独立验收各有边界



程序负责转换、校验与原子应用；业务含义及真实效果仍需独立验证

OntoWeave 的建模方法

先完成一条可读取、可验证的业务链，再扩展相邻范围

01

典型业务问题

“能申请，是否就能退款？”
这类问题称为能力问题 CQ

02

逻辑切片

按业务目标、生命周期与依赖
分步分析和验证

03

业务语义模型

先完成最小完整链
本项目用七视角检查覆盖

04

实际读取与回答

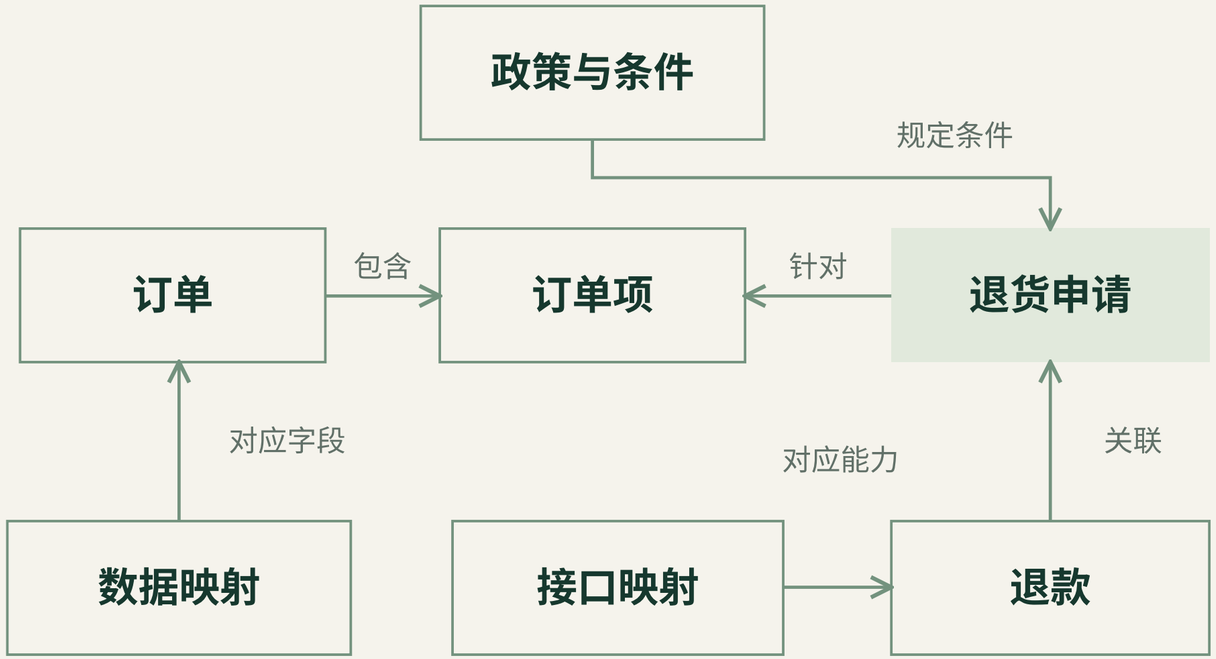
检查消费者是否正确使用
再扩展相邻切片

项目级始终保留：**共享术语、跨切片依赖、数据与接口映射、跨切片问题。**
读取和回答中的缺口，返回业务澄清与语义修订。

切片是组织分析与验证的方式；不按表数量直接决定有几个本体

OntoWeave 的七个建模视角

本项目用这七个视角组织与检查同一模型，可按场景取舍



01 对象	订单、订单项、商品、退货申请
02 行为	提交申请、审核、退款
03 规则	时间窗、适用范围、退款前提
04 角色	客户、客服、仓库
05 流程	申请、退回、验收、退款
06 数据映射	签收时刻 → orders.signed_at
07 接口映射	售后查询、申请提交、退款执行

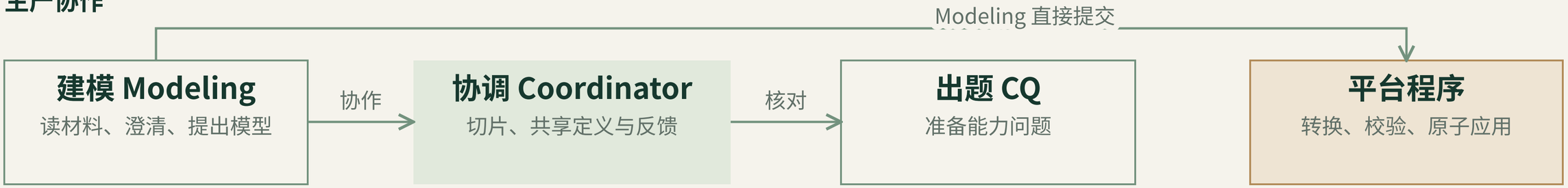
每个视角记录：已确认／待澄清／本范围不适用。

七视角是 OntoWeave 的组织与检查方式，不要求七个本体或七个 Agent

Agent Team 的自动建模方式

生产协作与独立验收分别承担职责，业务歧义仍需业务确认

生产协作



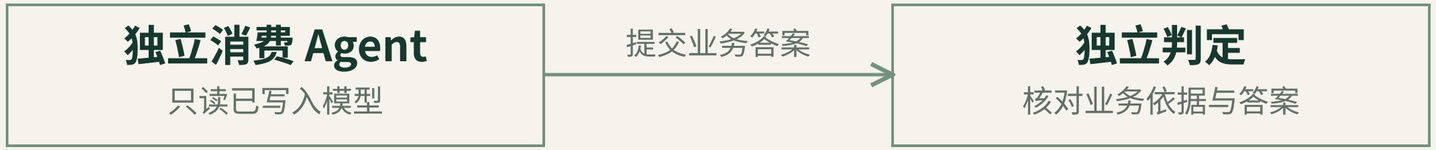
Modeling 直接提交变更 → 平台程序 → 回执 → 召回／周边自查

宿主监督 Delivery：授权、进度、交付与失败跟进

独立验收

公开反馈可用于修订

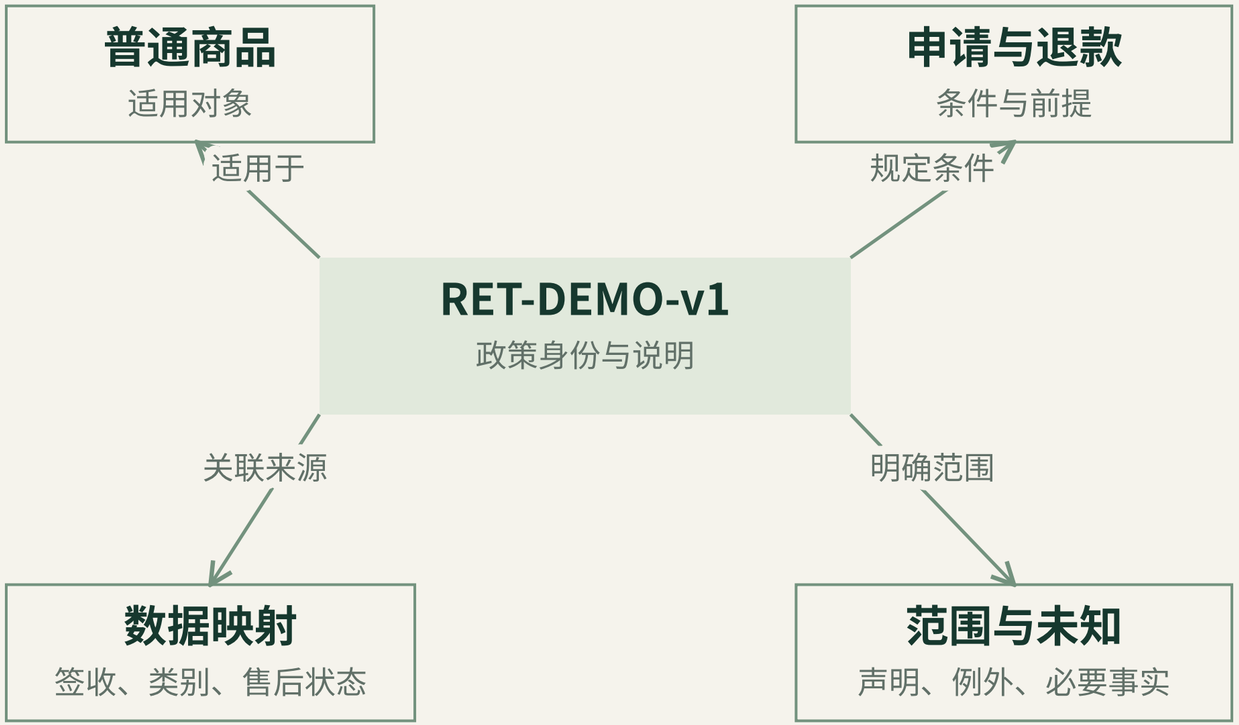
隐藏答案保持隔离



自动整理、建模和调用不替代业务确认；成功写入与业务语义通过分别证明

消费 Agent 实际需要读到什么

把“语义上下文”展开为能支撑业务判断的内容



RET-DEMO-v1

适用对象	普通商品，单件整件退货
申请条件	客户声明未使用 0 ⩽ 间隔 ⩽ 168 小时
退款前提	退回验收通过、客服批准
事实来源	`orders.signed_at` 客户输入、外部售后接口
范围与状态	按权限读取，检查分页／截断 缺必要信息时补读或询问

按权限读到必需内容，并检查分页与截断；读取状态不保证答案正确

虚构内容与展示形式，非真实接口响应

当前能力与验证边界

截至 2026-10-14，按需求及保留交付记录陈述

已实现能力

当前需求与交付范围

多视角语义表达、共享定义、确定性写入、受治理读取，
以及数据与接口映射表达。**工程检查通过，不等于业务效果已证明。**

已有有界证据

2026-10-06 · 工业服务四切片

冻结题集独立验收：**公开 18/18、隐藏 8/8**。
仅覆盖当时获准模型与题集，不能作为当前通用准确率。

仍待验证范围

10 月 13—14 日后续场景

后续运行存在未通过或未完成验收，R4.0-009 整体仍为 **PARTIAL**。
全范围、SQL/API 联合验收、跨场景重复性与产品化尚未完成。

当前不提供 OWL 自动推理、SHACL 专用校验、规则脚本或业务流程执行引擎。

历史有界通过保留原义，当前部分验证保留边界；本次没有重跑平台验收

第 4 章

电商退货与退款模拟案例

04

一家小型电商，希望让客服 Agent 判断退货资格，
并说明退款的下一步。

公开材料 ／ 确认口径 ／ 建立语义 ／ 取得事实 ／ 解释与验证

模拟业务背景与输入材料

A 店 · 单件商品整件退货 · 商品款整额退款

政策与业务确认

术语解释

数据库字段与编码说明

售后接口合同

客户本次声明

不涉及运费、促销分摊、部分退货、
跨境业务或法律解释。

三个对照订单

A101	10-01 10:00 签收 普通商品，声明未使用
A102	09-27 10:00 签收 普通商品，声明未使用
A103	签收时刻缺失 普通商品，声明未使用

拟申请／判断时点：**2026-10-06 10:00**
全部时间为 UTC+08:00

先公开范围、政策与事实来源，后续每个判断才能逐项核对

全部材料与订单均为虚构

建模前的业务确认

字段名和样例值是映射线索，业务确认决定政策含义

01

“七天内”

需要问：从哪一刻起算？
按日历日还是连续小时？

确认：提交减签收，
0—168 小时，包含等号。

02

“没使用就能退”

需要问：申请时怎样确认？
定制商品是否适用？

确认：普通商品，
申请时采用客户未使用声明。

03

“申请后退款”

需要问：还有哪些前提？
信息缺失怎样处理？

确认：验收通过、客服批准。
缺必要事实时保留未知。

客户声明与实际验收分别记录；申请受理、具备退款条件和退款完成分别判断

业务切片与共享定义

把 P21 的方法具体化为三段范围，逐步完成同一业务链

01

退货资格

确认适用商品、声明与时间窗。

首先回答：

是否允许提出申请?

02

退货处理

表达申请、退回与验收状态。

继续回答：

商品实际状态如何确认?

03

退款处理

表达审批、退款前提与接口。

跨环节回答：

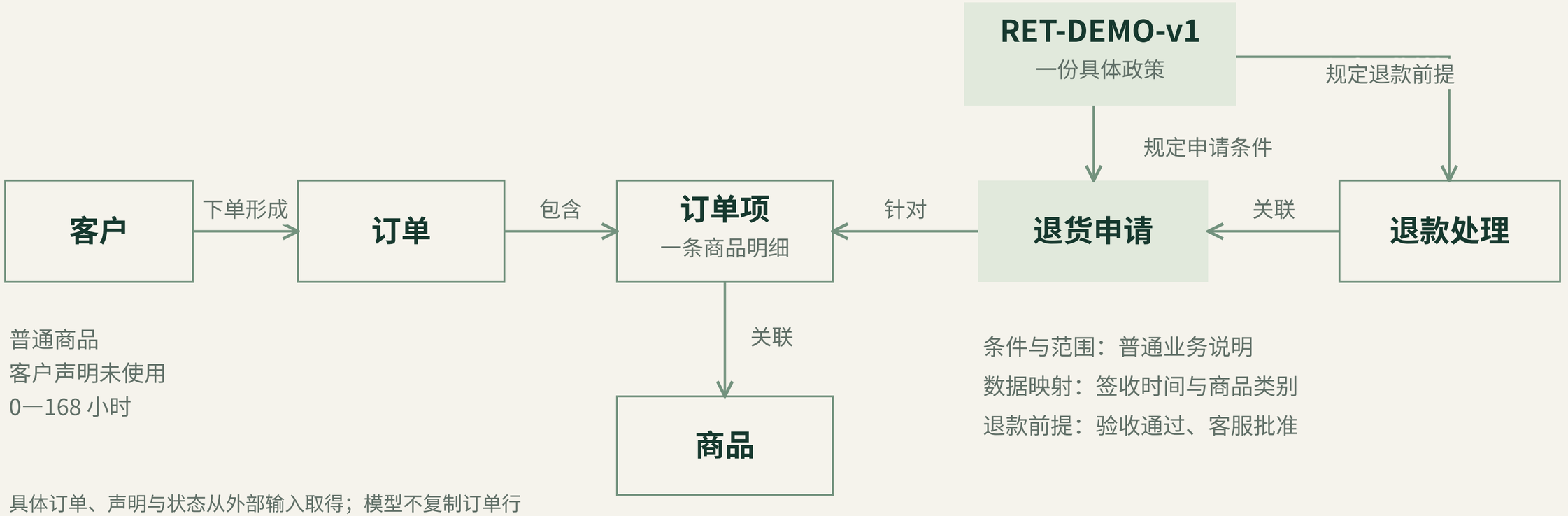
何时具备退款条件?

> 共同引用：**客户、订单项、商品、政策、状态含义，以及必要数据与接口映射。**
> 首条链也需保留“可以申请与退款完成是不同阶段”这一共同语义。

三个逻辑切片是分析组织方式，不预设三套本体或三个 Agent

退货与退款的语义模型

沿用 P11 的骨架，补上已经确认的条件、前提与映射



关系用于定位相关知识，普通业务说明保留条件、范围、例外和未知

语义怎样连接 SQL 与 API

先按获准语义取得事实，再由外部系统控制业务执行

查询事实

签收时刻 → `orders.signed_at`

商品类别 → `products.category`

订单项通过 `order_id` 与 `product_id`
连接订单和商品。语义映射约束 SQL。

> 声明来自**客户本次输入**。
> 验收、审批与退款状态来自**外部售后查询接口**。

执行动作

提交申请 → `POST /returns`

执行退款 → `POST /refunds`

外部系统检查条件、业务权限与防重复。
语义知识中的前提，不授予操作权限。

> `ACCEPTED` 只表示退款请求受理。
> `SUCCEEDED` 才表示系统确认完成。

SQL、时间计算工具和 API 都是教学设计，本次没有接入或调用退款系统

案例一：能申请，为什么不能马上退款

A101 · RET-DEMO-v1 · 2026-10-06 10:00+08:00 的模拟判断

RET-DEMO-v1

适用对象	普通商品，单件整件退货
申请条件	客户声明未使用 0 ⩽ 间隔 ⩽ 168 小时
退款前提	退回验收通过、客服批准
事实来源	`orders.signed_at` 客户输入、外部售后接口

取得具体事实

订单库	10-01 10:00 签收 普通商品
客户输入	声明未使用
计算工具	间隔 **120 小时** 0 ⩽ 120 ⩽ 168
售后接口	验收尚未开始 审批尚未申请 无退款记录

按当前政策和您的未使用声明，可以申请退货。

实际商品状态仍待验收。

当前尚未验收，也尚未获客服批准，**不能立即退款。**

下一步：如决定申请，由业务系统在实际提交时重检条件与权限。

可申请、已提交、具备退款条件、退款完成是不同阶段

案例二：超时与信息不足

相同政策，对明确不满足与无法判断分别回答

A102 · 超时

216 小时

09-27 10:00 签收，10-06 10:00 拟申请。
普通商品，客户声明未使用。

超过 168 小时，
不符合本例政策时间条件。

解释明确不满足的条件，
不编造退款阶段事实。

A103 · 必要信息缺失

未知

签收时刻缺失，无法计算间隔。
普通商品，客户声明未使用。

暂无法判定时间条件，
需要补充签收时刻。

保留未知并提出补问，
不将未知改写成允许或拒绝。

明确不满足的条件可以支持否定结论；必要条件尚未确定时保留相应未知

怎样验证建模效果

独立检查含义、实际读取与业务回答，避免把写入成功当作质量证明

资料依据

先确定能力问题、适用范围
和可核对的判定依据

模型表达

概念、关系、条件、例外
与未知是否忠于资料

实际检索

在真实授权接口读取
必需内容是否取得

业务答案

独立消费 Agent 回答
独立判定核对依据

同条件比较

RAG 与本体辅助方案共用相同资料、澄清结果、政策版本、数据快照、题目、工具与权限。
尽量固定语言模型及调用条件，改变知识组织与读取方式。

A101：可申请，不能立即退款　A102：超时　A103：补问　168 小时：含等号

分别记录错误允许、错误拒绝、应补问却强答与合理未知，以及建模、维护、确认和运行成本。

显式语义连接依据，外部事实支持结论，业务系统控制执行，独立验证检查整条链

FAQ：什么时候值得使用本体

先看当前任务与已有方案，再决定是否增加显式语义建模

已经有 RAG，还需要本体吗？

文档检索已能稳定回答时，可以继续使用。跨对象、跨任务的关系需要明确复用时，可试用本体并做同条件比较。

直接把业务规则写进提示词不行吗？

小范围任务可以。文档、词典和规则服务也能共享口径。显式关联带来的复用收益值得投入时，再进一步建模。

本体会替代数据库或业务系统吗？

业务记录与操作仍留在原系统。本体提供业务含义、关系、条件及系统映射。

所有业务都要先建本体吗？

优先选择含义歧义多、跨系统关系多、同一定义被反复使用的场景。简单固定任务可以采用更轻的方案。

本体与知识图谱有什么区别？

本体明确概念、含义和关系；知识图谱用实体与关系组织知识，可使用或包含本体定义。两者可以结合。

回看方法局限 P07　回看概念分工 P13　回看适用条件 P17

从一个有代表性的跨环节问题试起，比较正确性与实际投入

FAQ：怎么建，哪些工作需要人参与

从一条有界业务链开始，按实际依赖安排分工

必须先学会 RDF 和 OWL 吗?

业务人员先确认概念与口径，技术人员选择表达及查询方式。理解本体不要求先写 OWL，当前项目也不要求自动推理。

七个视角是通用的本体方法论吗?

这是 OntoWeave 当前的组织与检查方式，可按场景取舍；其他方案可以不同，也不要求七个本体或 Agent。

Agent 能全自动把模型建好吗?

可以自动整理资料、提出模型并调用平台。资料缺失、口径冲突与授权仍需确认，质量由独立验证判断。

是否需要一次建完整个企业?

先完成一条有代表性、可验证的业务链，再维护共享定义，并逐步扩展相邻切片。

建模方法 P21　七个视角 P22　Team 分工 P23

自动化承担整理与调用，业务人员确认含义，独立验证判断是否可用

FAQ：怎样保证质量并持续维护

定义、事实、检索和使用都需要检查；政策变更还需外部系统对齐

有了本体就不会产生幻觉了吗？

源材料、模型、检索与 Agent 使用都可能出错。需要逐环节验证，也允许合理回答未知。

业务规则变了，是否要全部重建？

业务负责人确认新口径、生效时间和范围，再修订相关定义及映射，检查受影响问题。修改范围由实际依赖决定。

怎样评估成本与收益？

同资料、同澄清、同工具比较正确性和完整性，分别记录建模、维护、人工确认与调用成本。

本体权限能保护数据库和退款吗？

语义权限控制语义知识。数据库行列访问与退款操作权限由外部系统执行，必须分别验证。

政策变更示例

168 → 240 小时

自 **2026-11-01 00:00+08:00**
起提交的申请采用 v2。
此前提交的仍按 v1。

业务负责人确认
维护者修订语义
系统负责人对齐执行
独立验证新旧边界

独立教学变更，不改变前三例。
不代表自动影响分析或任意历史版本读取功能。

结合你的业务继续讨论：哪些含义、关系与判断，最需要共同引用？